

Putting Things in Order -- Why Sorting is Everywhere

Lecture 21

Data Structures and Algorithms

Where We Are in the Course

We have now studied how to **store** and **retrieve** data efficiently:

- Arrays, Linked Lists, Stacks, Queues
- Trees, Heaps, Hash Tables, Graphs

But one of the most important questions in computing is missing:

■ **"How do we arrange data so that it can be used efficiently?"**

This new unit answers that question. We study **Sorting and Searching** -- the algorithms that sit at the heart of almost every software system ever built.

Why Does Sorting Matter?

Think about how often you encounter sorted data every day:

- Your phone contacts -- sorted alphabetically
- Your email inbox -- sorted by date
- An e-commerce site showing products by price
- A leaderboard ranked by score
- A dictionary -- sorted alphabetically for fast lookup

Sorting is the prerequisite for many other algorithms. Binary search (which we study later) requires sorted data. Merge algorithms need sorted inputs. Database joins are vastly faster on sorted tables.

Donald Knuth dedicated an entire volume of *The Art of Computer Programming* to sorting -- a testament to how central it is.

Part 1: Key Properties of Sorting Algorithms

Two Questions Every Sorting Algorithm Must Answer

Before we study any specific algorithm, we need a vocabulary to describe and compare them.

There are two fundamental properties that every sorting algorithm has:

1. **Is it stable?**

-- Does the algorithm preserve the relative order of equal elements?

2. **Is it in-place?**

-- Does the algorithm sort without needing a large amount of extra memory?

These two properties determine which algorithm is right for a given situation. Let us understand them clearly.

Property 1: Stability

Definition: A sorting algorithm is **stable** if equal elements remain in the same relative order after sorting as they were before.

This matters more than it first appears. Imagine sorting a list of students by grade:

Before sorting (original order):

(Alice, 85) (Bob, 90) (Carol, 85) (Dave, 90)

Stable sort by grade:

(Alice, 85) (Carol, 85) (Bob, 90) (Dave, 90)

^ Alice still comes before Carol (they had equal grades)

Unstable sort by grade:

(Carol, 85) (Alice, 85) (Dave, 90) (Bob, 90)

^ Relative order of 85s has changed -- Carol jumped ahead of Alice

If you sort first by grade and then by name, a **stable** name sort preserves the grade ordering within each name group.

Stability: A Playing Card Analogy

Imagine you have a deck of cards and you want to sort them **by rank only** (ignoring suit).

After sorting, if two cards have the same rank (e.g., two Kings), a **stable** sort guarantees they appear in the same left-to-right order as they were originally in the deck.

An **unstable** sort might flip their order.

This distinction is critical in database systems where records often need to be sorted by multiple fields in sequence.

Stability ensures earlier sorts are respected.

Which algorithms are stable?

- Stable: Bubble Sort, Insertion Sort, Merge Sort
- Unstable: Selection Sort, Quicksort (standard), Heap Sort

Property 2: In-Place Sorting

Definition: A sorting algorithm is **in-place** if it uses only a **constant amount of extra memory** -- $O(1)$ additional space beyond the input array itself.

In-place algorithms rearrange elements within the original array using swaps. They do not create a separate copy of the data.

In-Place Sort:

Input array: [5, 3, 8, 1, 4]

Sorting... (only swaps happen within this same array)

Output: [1, 3, 4, 5, 8] <-- same array, rearranged

NOT In-Place (e.g., standard Merge Sort):

Input array: [5, 3, 8, 1, 4]

Temp array: [_, _, _, _, _] <-- needs $O(n)$ extra memory

Merge into: [1, 3, 4, 5, 8]

In-place: Bubble Sort, Selection Sort, Insertion Sort, Quicksort, Heap Sort

Not in-place: Standard Merge Sort -- needs $O(n)$ extra space

Property 3: Internal vs. External Sorting

There is a third classification that depends not on the algorithm's behavior but on the **size of the data** relative to memory.

Internal Sorting

- The **entire dataset fits in RAM** (main memory)
- Random access to any element is instant
- Goal: minimize comparisons and element movements
- **All algorithms we study in detail are internal sorting algorithms**

External Sorting

- The dataset is **too large to fit in RAM** -- it lives on a hard disk or SSD
- Disk access is 10,000 to 100,000 times slower than RAM access
- Goal: minimize the number of disk read/write operations
- Strategy: load small chunks into memory, sort them, write back, then **merge** the sorted chunks

External Sorting: The Big Picture

When you need to sort a 100 GB log file on a machine with 4 GB of RAM, internal sorting is impossible. External sorting handles this with a two-phase strategy:

```
Phase 1 -- Sort Chunks (Runs):  
  Read chunk 1 (fits in RAM) -> Sort it -> Write sorted chunk to disk  
  Read chunk 2 (fits in RAM) -> Sort it -> Write sorted chunk to disk  
  ... repeat for all chunks ...  
  
Phase 2 -- Merge the Sorted Chunks:  
  Chunk 1: [1, 5, 9, 13 ...]  
  Chunk 2: [2, 4, 8, 11 ...] ---> Merge into one sorted output  
  Chunk 3: [3, 6, 7, 10 ...]
```

This technique is called **External Merge Sort**. Notice: it uses merge sort's merge step as its core idea. This is exactly why merge sort is important -- it is the backbone of real-world large-scale data processing.

Internal vs. External: Summary

Feature	Internal Sorting	External Sorting
Data location	All in RAM	On disk / SSD
Element access	Instant (nanoseconds)	Slow (milliseconds)
Primary concern	Fewer comparisons / swaps	Fewer disk I/O operations
Example algorithms	Bubble, Insertion, Merge, Quick	External Merge Sort
Typical use case	Arrays up to a few million items	Files of gigabytes or more

Connection to previous units: Remember B-Trees? They were designed to minimize disk accesses for database indexing. External sorting follows the same philosophy -- disk I/O is the bottleneck, so we minimize it at all costs.

Part 2: Bubble Sort

"The Simplest Sort -- Watching Bubbles Rise"

The Bubble Sort Idea

Bubble Sort is named for the way **large elements bubble up** toward the end of the array with each pass.

The Strategy:

- Repeatedly walk through the array from left to right
- Compare each adjacent pair of elements
- If they are in the wrong order, **swap** them
- After one full pass, the **largest element is guaranteed to be at the end**
- Repeat, but now ignore the last position (it is already sorted)
- After each pass, the sorted region at the right grows by one

Analogy: Imagine bubbles in a glass of soda. The largest bubble rises to the top first. Then the next largest rises. The array sorts itself from right to left, one element per pass.

Bubble Sort: The Algorithm

```
BubbleSort(A, n):  
    for i = 0 to n-2:                // n-1 passes needed  
        for j = 0 to n-2-i:          // inner loop shrinks each pass  
            if A[j] > A[j+1]:  
                swap(A[j], A[j+1])
```

What each line does:

- **Outer loop (i)**: Controls how many passes we make. After pass **i**, the last **i+1** elements are sorted.
- **Inner loop (j)**: Walks through the unsorted portion, comparing adjacent elements.
- **n-2-i upper bound**: We stop earlier each pass because the right side is already sorted -- no need to check it again.

Bubble Sort: Complete Trace

Array: [64, 25, 12, 22, 11]

Pass 1 (i=0): Compare all adjacent pairs

```
[64, 25, 12, 22, 11]
  ^   ^
64 > 25? Yes -> swap -> [25, 64, 12, 22, 11]
    ^   ^
64 > 12? Yes -> swap -> [25, 12, 64, 22, 11]
      ^   ^
64 > 22? Yes -> swap -> [25, 12, 22, 64, 11]
        ^   ^
64 > 11? Yes -> swap -> [25, 12, 22, 11, 64]
                                   ^^^^
                                   64 is sorted!
```

After Pass 1: [25, 12, 22, 11, | 64] -- the largest element (64) has bubbled to the end.

Bubble Sort: Passes 2, 3, and 4

Pass 2 (i=1): Compare up to position 3

```
[25, 12, 22, 11, | 64]
25>12? swap -> [12, 25, 22, 11, | 64]
25>22? swap -> [12, 22, 25, 11, | 64]
25>11? swap -> [12, 22, 11, |25, 64|]
                      ^^^^^^^^
                      25 and 64 are sorted!
```

Pass 3 (i=2): Compare up to position 2

```
[12, 22, 11, | 25, 64]
12>22? No.
22>11? swap -> [12, 11, |22, 25, 64|]
```

Pass 4 (i=3): Compare up to position 1

```
[12, 11, | 22, 25, 64]
12>11? swap -> [|11, 12, 22, 25, 64|]
```

Result: [11, 12, 22, 25, 64] -- sorted!

Bubble Sort: The Sorted Boundary

A useful way to visualize bubble sort is tracking the growing sorted zone on the right:

```
Start:    [64, 25, 12, 22, 11]
Pass 1:   [25, 12, 22, 11] | 64 |
Pass 2:   [12, 22, 11] | 25, 64 |
Pass 3:   [12, 11] | 22, 25, 64 |
Pass 4:   [11] | 12, 22, 25, 64 |
Done:     | 11, 12, 22, 25, 64 |
```

After each pass, one more element settles into its final position on the right. The unsorted region shrinks by one each time.

This is the key insight: **Bubble Sort makes at most $n - 1$ passes**, and in each pass, the inner loop does one less comparison than the previous pass.

Optimization: Early Termination

Consider this array: [1, 2, 3, 5, 4]

Standard bubble sort would still make 4 passes even though the array is almost sorted.

Improvement: Add a flag `swapped`. If an entire pass completes with **no swaps**, the array is already sorted -- stop immediately.

```
BubbleSort_Optimized(A, n):  
    for i = 0 to n-2:  
        swapped = false  
        for j = 0 to n-2-i:  
            if A[j] > A[j+1]:  
                swap(A[j], A[j+1])  
                swapped = true  
        if swapped == false:  
            break           // No swaps this pass -- already sorted!
```

With this optimization, sorting an already-sorted array takes only **one pass** with zero swaps -- $O(n)$ in the best case.

Bubble Sort: Complexity Analysis

Case	Comparisons	Swaps	Time
Best case (sorted, with flag)	$n - 1$	0	$O(n)$
Average case	$\frac{n(n-1)}{4}$	$\frac{n(n-1)}{4}$	$O(n^2)$
Worst case (reverse sorted)	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$	$O(n^2)$
Space	--	--	$O(1)$

Properties

- **In-place:** Yes -- only a single temporary variable is used for swapping
- **Stable:** Yes -- we only swap when $A[j] > A[j+1]$ (strict greater than). Equal elements are never swapped, so their relative order is preserved.
- **Adaptive:** With the early termination flag, performs well on nearly-sorted input

Bubble Sort: When Does It Hit Worst Case?

The worst case occurs on a **reverse-sorted array** -- every pair is in the wrong order, so every comparison leads to a swap.

Example: [5, 4, 3, 2, 1]

- Pass 1: 4 swaps
- Pass 2: 3 swaps
- Pass 3: 2 swaps
- Pass 4: 1 swap
- Total: **10 swaps** = $\frac{5 \times 4}{2} = \frac{n(n-1)}{2}$

For $n = 1000$: up to **499,500 swaps**.

For $n = 1,000,000$: up to **499,999,500,000 swaps** -- clearly impractical.

Conclusion: Bubble Sort is easy to understand and implement, but is impractical for large datasets. Its real value is pedagogical -- understanding it builds intuition for better algorithms.

Part 3: Stability -- Seeing It in Action

Why Stability Matters: A Multi-Key Sort

Suppose we have a list of student records and we want to sort by **grade first, then by name within the same grade**.

A common technique: sort by name first (alphabetically), then sort by grade. If the grade sort is **stable**, it preserves the name ordering within each grade.

Step 1: Sort by name (any stable sort):

```
(Alice, 85) (Bob, 90) (Carol, 85) (Dave, 90)
```

Step 2: Sort by grade using a STABLE sort:

```
(Alice, 85) (Carol, 85) (Bob, 90) (Dave, 90)
```

```
^-- Alice still before Carol (name order preserved within grade 85)
```

Step 2: Sort by grade using an UNSTABLE sort:

```
(Carol, 85) (Alice, 85) (Dave, 90) (Bob, 90)
```

```
^-- Name order within grade 85 is LOST
```

Stability is what makes multi-key sorting work correctly. Database `ORDER BY grade, name` relies on this exact principle.

Verifying Bubble Sort's Stability

Consider: **[3a, 1, 3b, 2]** (subscripts show original order of equal elements)

Pass 1:

[3a, 1, 3b, 2]

3a > 1? Yes -> swap -> [1, 3a, 3b, 2]

3a > 3b? No (equal, strict > fails) -> no swap -> [1, 3a, 3b, 2]

3b > 2? Yes -> swap -> [1, 3a, 2, 3b]

Pass 2:

[1, 3a, 2, 3b]

1 > 3a? No. 3a > 2? Yes -> swap -> [1, 2, 3a, 3b]

Pass 3:

[1, 2, 3a, 3b] -- No swaps. Done.

Result: [1, 2, 3a, 3b]

The two 3s remain in their original order (3a before 3b). Bubble Sort is stable because it uses **>** (strict greater than) -- equal elements are never swapped.

Part 4: Summary and What is Next

Today's Key Concepts

Three fundamental properties of any sorting algorithm:

1. **Stability** -- Preserves relative order of equal elements
 - Critical for multi-key sorting and database operations
2. **In-place** -- Sorts using only $O(1)$ extra memory
 - Important when memory is constrained
3. **Internal vs. External** -- Dictated by whether data fits in RAM
 - External sorting uses chunk-sort-merge strategy (External Merge Sort)

Bubble Sort:

- Simple, intuitive, stable, in-place
- $O(n^2)$ average and worst case; $O(n)$ best case with early termination flag
- Important for understanding -- not used in practice for large data

Complexity Summary So Far

Algorithm	Best	Average	Worst	Space	Stable?	In-Place?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes

We will fill in this table over the next four lectures as we study Selection Sort, Insertion Sort, Merge Sort, and Quicksort. By the end of this unit, you will be able to recommend the right algorithm for any sorting scenario.

Quick Check: Test Your Understanding

Before next class, try to answer these:

1. An array is sorted in reverse order. How many passes does bubble sort (without flag) make?
2. An array is already sorted. How many passes does bubble sort **with** the early termination flag make?
3. You have a list of (employee_id, salary) pairs. You want to sort by salary, and for equal salaries, preserve the original order of employees. Which property of a sorting algorithm do you need?
4. A company needs to sort a 500 GB transaction log file using a machine with 8 GB of RAM. Should they use internal or external sorting? What algorithm would you suggest?

Visual Tool: See Bubble Sort in Action

VisuAlgo -- Sorting Algorithm Visualizer:

<https://visualgo.net/en/sorting>

Select "Bubble Sort" from the menu. You can:

- Set your own input array or generate a random one
- Step through the algorithm one comparison at a time
- Watch elements swap and the sorted boundary grow from the right
- Switch between algorithms to compare how different strategies sort the same array

Try sorting the same array with Bubble Sort, then with other algorithms we will study next -- you will see a dramatic difference in the number of operations.

What Comes Next

Lecture 22 continues with two more $O(n^2)$ algorithms:

- **Selection Sort** -- Find the minimum in the unsorted region, swap it to the front. Simple but unstable. Has only $n - 1$ swaps (minimum among simple sorts).
- **Insertion Sort** -- Build the sorted array one card at a time, inserting each new element into its correct position. Stable, adaptive, the best simple sort in practice.

We will then compare all three head-to-head and ask the obvious question: **can we do better than $O(n^2)$?**

The answer is yes -- and the proof of the limit (known as the $\Omega(n \log n)$ lower bound) is one of the most elegant results in computer science. Stay tuned.

